

**name:** Datadog SAST and Security Gate

**on:**

**push:**

**branches:** [ main, master, develop ]

**pull\_request:**

**branches:** [ main, master, develop ]

**jobs:**

**datadog-sast-check:**

**runs-on:** ubuntu-latest

**steps:**

# 1. Checkout repository with full git history

- **name:** Checkout Code

**uses:** actions/checkout@v4

**with:**

**fetch-depth:** 0 # Required for Datadog to accurately track Git commit history

# 2. Run Datadog Static Analysis

# This action automatically scans your code and uploads the results to Datadog

- **name:** Run Datadog Static Analyzer

**id:** datadog-static-analysis

**uses:** DataDog/datadog-static-analyzer-github-action@v3

**with:**

**dd\_api\_key:** \${{ secrets.DD\_API\_KEY }}

**dd\_app\_key:** \${{ secrets.DD\_APP\_KEY }}

**dd\_site:** "datadoghq.com" # Change to "datadoghq.eu" if your account is in Europe

**cpu\_count:** 2

**enable\_performance\_statistics:** false

### # 3. Trigger Datadog Quality Gates

# This queries Datadog's API and returns exit code 1 if the newly found issues break your rules

- **name:** Evaluate Datadog Security Gate

**uses:** DataDog/datadog-ci-action@v1

**env:**

**DD\_API\_KEY:** \${{ secrets.DD\_API\_KEY }}

**DD\_APP\_KEY:** \${{ secrets.DD\_APP\_KEY }}

**DD\_SITE:** "datadoghq.com"

**with:**

**command:** gate evaluate

### Next Steps to Make This Work:

1. **Add Secrets to GitHub:** Go to the repository settings under **Settings > Secrets and variables > Actions** and add the following:
  - a. **DD\_API\_KEY:** Your unique Datadog API key.
  - b. **DD\_APP\_KEY:** Your Datadog Application key  
(requires `code_analysis_read` and `quality_gate_read` permissions).  
the **DD\_APP\_KEY** secret must have the specific **code\_analysis\_read** scope enabled within the Datadog Organization settings.
  - c. **DD\_SITE:** Your Datadog region parameter. Defaults to `datadoghq.com`, but change it to `datadoghq.eu` or `://datadoghq.com` depending on where your account is hosted
2. **Define Rules in Datadog:** Ensure you have configured a rule in your Datadog UI under **Software Delivery > Quality Gates** that specifies what to block (e.g., Block if New Vulnerabilities introduced is  $\geq 1$  and Severity = Critical).  
If a critical code vulnerability is found, Datadog sets this status check to **Failed**.
3. Make sure that the branch protection rules require all status checks to pass, the **"Merge" button is grayed out and blocked** for developers until the vulnerability is resolved.

**Reference:**

[https://docs.datadoghq.com/security/code\\_security/static\\_analysis/setup/github\\_actions/](https://docs.datadoghq.com/security/code_security/static_analysis/setup/github_actions/)

[https://docs.datadoghq.com/security/code\\_security/static\\_analysis/](https://docs.datadoghq.com/security/code_security/static_analysis/)

<https://github.com/DataDog/datadog-static-analyzer-github-action>

<https://github.com/DataDog/datadog-sca-github-action>

[https://docs.datadoghq.com/source\\_code/](https://docs.datadoghq.com/source_code/)

[https://docs.datadoghq.com/source\\_code/source-code-management](https://docs.datadoghq.com/source_code/source-code-management)

<https://docs.datadoghq.com/integrations/github/>